Lesson 3

Material

https://github.com/unixfile/learning-python-2026

Today

- Lists: creation, indexing, methods
- Slicing
- Copying lists
- Multidimensional lists
- Mutability vs. immutability
- Tuples
- for loops and enumerate
- break and continue
- while loops

1 Creating a list

```
In : fruits = ["banan", "melon", "kiwi", "citron"]
In : type(fruits)
Out: list
In : len(fruits)
Out: 4
```

2 Indexing

```
In : fruits = ["banan", "melon", "kiwi", "citron"]
In : fruits[0]
Out: 'banan'

In : fruits[-1]  # -1 is the last element
Out: 'citron'
```

3 Modifying elements

```
In : fruits = ["banan", "melon", "kiwi", "citron"]
In : fruits[0] = "apelsin"
In : fruits
Out: ['apelsin', 'melon', 'kiwi', 'citron']
```

4 List methods

```
In : fruits = ["banan", "melon", "kiwi", "citron"]
In : fruits.append("mango")
In : fruits.pop()
Out: 'mango'

In : fruits.insert(0, "vindruva")
In : fruits.remove("kiwi")
In : fruits
Out: ['vindruva', 'banan', 'melon', 'citron']
```

5 Sorting

```
In : nums = [3, 1, 4, 1, 5, 9, 2, 6]
In : nums.sort()
In : nums
Out: [1, 1, 2, 3, 4, 5, 6, 9]

In : sorted([3, 1, 4])
Out: [1, 3, 4]
```

- `sort()` modifies in place; `sorted()` returns a new list

6 Slicing

```
In : s = [0, 1, 2, 3, 4, 5]
In : s[1:4]  # includes 1, excludes 4
Out: [1, 2, 3]
In : s[:3]
Out: [0, 1, 2]
In : s[3:]
Out: [3, 4, 5]
In : s[::2]
Out: [0, 2, 4]
In : s[::-1]
Out: [5, 4, 3, 2, 1, 0]
```

7 Copying lists

```
In : a = [1, 2, 3]
In : b = a
In : b.append(4)
In : a
Out: [1, 2, 3, 4]

In : c = a.copy()
In : c.append(5)
In : a
Out: [1, 2, 3, 4]
```

- Assignment copies the reference, not the list

8 Multidimensional lists

```
In : matrix = [[1, 2, 3],
   :           [4, 5, 6],
   :           [7, 8, 9]]
In : matrix[1][2]
Out: 6
```

9 Mutability

```
# Lists are mutable
numbers = [1, 2, 3]
numbers[0] = 99  # ok

# Strings are immutable
s = "hello"
s[0] = "H"  # TypeError
```

10 Tuples

```
In : t = (1, 2, 3)
In : type(t)
Out: tuple

In : t[0]
Out: 1

In : t[0] = 99  # TypeError: object does not support item a
```

11 Tuples, continued...

```
In : point = (10, 20)
In : x, y = point
In : x
Out: 10

In : single = (42,)
In : type(single)
Out: tuple
```

- Use tuples for fixed data, lists for dynamic data

12 for loop

```
fruits = ["banan", "melon", "kiwi", "citron"]

for fruit in fruits:
    print(fruit)
```

- Think of `for` as “for each”
- Works on any iterable: lists, strings, files, . . .

13 range()

```
In : list(range(5))
Out: [0, 1, 2, 3, 4]

In : list(range(2, 8))
Out: [2, 3, 4, 5, 6, 7]

In : list(range(0, 10, 2))
Out: [0, 2, 4, 6, 8]
```

14 break and continue

```
# break exits the loop early
for n in range(1, 10):
    if n == 5:
        break
    print(n, end=' ')
# Output: 1 2 3 4

# continue skips to the next iteration
for n in range(1, 10):
    if n % 2 == 0:
        continue
    print(n, end=' ')
# Output: 1 3 5 7 9
```

15 enumerate

```
fruits = ["banan", "melon", "kiwi", "citron"]

for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# Output:
# 0: banan
# 1: melon
# 2: kiwi
# 3: citron
```

16 while loop

```
i = 5
while i > 0:
    print(i, end=' ')
    i -= 1
# Output: 5 4 3 2 1
```

- Loop runs as long as the condition is `True`
- Forgetting to update `i` creates an infinite loop

17 while True

```
while True:
    user_input = input("Enter 'exit' to stop: ")
    if user_input == 'exit':
        break
    print(f"You entered: {user_input}")
```

- `break` is the only way out of a `while True:` loop
- Useful for menus and simple REPLs